

BASICS OF HADOOP**6**

Data format – analyzing data with Hadoop – scaling out – Hadoop streaming – Hadoop pipes – design of Hadoop distributed file system (HDFS) – HDFS concepts – Java interface – data flow – Hadoop I/O – data integrity – compression – serialization – Avro – file-based data structures – Cassandra – Hadoop integration.

4.1 BASICS OF HADOOP - DATA FORMAT

Hadoop is an open-source framework for processing, storing, and analyzing large volumes of data in a distributed computing environment. It provides a reliable, scalable, and distributed computing system for big data.

Key Components:

- **Hadoop Distributed File System (HDFS):** HDFS is the storage system of Hadoop, designed to store very large files across multiple machines.
- **MapReduce:** MapReduce is a programming model for processing and generating large datasets that can be parallelized across a distributed cluster of computers.
- **YARN (Yet Another Resource Negotiator):** YARN is the resource management layer of Hadoop, responsible for managing and monitoring resources in a cluster.

Advantages of Hadoop:

- **Scalability:** Hadoop can handle and process vast amounts of data by distributing it across a cluster of machines.
- **Fault Tolerance:** Hadoop is fault-tolerant, meaning it can recover from failures, ensuring that data processing is not disrupted.
- **Cost-Effective:** It allows businesses to store and process large datasets cost-effectively, as it can run on commodity hardware.

Here are some basics:

1. Data Storage in Hadoop:

- Hadoop uses the Hadoop Distributed File System (HDFS) to store data across multiple machines in a distributed fashion. Data is divided into blocks (typically 128 MB or 256 MB in size), and each block is replicated across several nodes in the cluster for fault tolerance.

2. Data Formats:

- Hadoop can work with various data formats, but some common ones include:
 - **Text:** Data is stored in plain text files, such as CSV or TSV.
 - **SequenceFile:** A binary file format optimized for Hadoop, suitable for storing key-value pairs.
 - **Avro:** A data serialization system that supports schema evolution. It's often used for complex data structures.

- **Parquet:** A columnar storage format that is highly optimized for analytics workloads. It's efficient for both reading and writing.
- 3. **Data Ingestion:**
 - Before analyzing data, you need to ingest it into Hadoop. You can use tools like Apache Flume, Apache Sqoop, or simply copy data into HDFS using Hadoop commands.
- 4. **Data Processing:**
 - Hadoop primarily processes data using a batch processing model. It uses a programming model called MapReduce to distribute the processing tasks across the cluster. You write MapReduce jobs to specify how data should be processed.
 - In addition to MapReduce, Hadoop ecosystem also includes higher-level processing frameworks like Apache Spark, Apache Hive, and Apache Pig, which provide more user-friendly abstractions for data analysis.
- 5. **Data Analysis:**
 - Once data is processed, you can analyze it to gain insights. This may involve running SQL-like queries (with Hive), machine learning algorithms (with Mahout or Spark MLlib), or custom data processing logic.
- 6. **Data Output:**
 - After analysis, you can store the results back into Hadoop, or you can export them to other systems for reporting or further analysis.
- 7. **Data Compression:**
 - Hadoop allows data compression to reduce storage requirements and improve processing speed. Common compression formats include Gzip, Snappy, and LZO.
- 8. **Data Schema:**
 - When working with structured data, it's important to define a schema. Some formats like Avro and Parquet have built-in schema support. In other cases, you may need to maintain the schema separately.
- 9. **Data Partitioning and Shuffling:**
 - During data processing, Hadoop can partition data into smaller chunks and shuffle it across nodes to optimize the processing pipeline.
- 10. **Data Security and Access Control:**
 - Hadoop provides security mechanisms to control access to data and cluster resources. This includes authentication, authorization, and encryption.

4.1.1. Step-by-step installation of Hadoop on a single Ubuntu machine.

Installing Hadoop on a single-node cluster is a common way to set up Hadoop for learning and development purposes. In this guide, I'll walk you through the step-by-step installation of Hadoop on a single Ubuntu machine.

Prerequisites:

- A clean installation of Ubuntu.
- Java installed on your system.

Let's proceed with the installation:

Step 1: Download Hadoop

1. Visit the Apache Hadoop website (<https://hadoop.apache.org>) and choose the Hadoop version you want to install. Replace x.y.z with the version number you choose.
2. Download the Hadoop distribution using `wget` or your web browser. For example:

```
bash
wget https://archive.apache.org/dist/hadoop/common/hadoop-X.Y.Z/hadoop-X.Y.Z.tar.gz
```

Step 2: Extract Hadoop 3. Extract the downloaded Hadoop tarball to your desired directory (e.g., `/usr/local/`):

```
bash
sudo tar -xzvf hadoop-X.Y.Z.tar.gz -C /usr/local/
```

Step 3: Configure Environment Variables 4. Edit your `~/.bashrc` file to set up environment variables. Replace x.y.z with your Hadoop version:

```
bash
export HADOOP_HOME=/usr/local/hadoop-X.Y.Z
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

Apply these changes to your current shell:

```
bash
source ~/.bashrc
```

Step 4: Edit Hadoop Configuration Files 5. Navigate to the Hadoop configuration directory:

```
bash
cd $HADOOP_HOME/etc/hadoop
```

6. Edit the `hadoop-env.sh` file to specify the Java home directory. Add the following line to the file, pointing to your Java installation:

```
bash
export JAVA_HOME=/usr/lib/jvm/default-java
```

7. Configure Hadoop's `core-site.xml` by editing it and adding the following XML snippet. This sets the Hadoop Distributed File System (HDFS) data directory:

```
xml
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://localhost:9000</value>
</property>
```

8. Configure Hadoop's `hdfs-site.xml` by editing it and adding the following XML snippet. This sets the HDFS data and metadata directories:

```
xml
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>

<property>
  <name>dfs.namenode.name.dir</name>
  <value>file:///usr/local/hadoop-X.Y.Z/data/namenode</value>
</property>

<property>
  <name>dfs.datanode.data.dir</name>
  <value>file:///usr/local/hadoop-X.Y.Z/data/datanode</value>
</property>
```

Step 5: Format the HDFS Filesystem 9. Before starting Hadoop services, you need to format the HDFS filesystem. Run the following command:

```
bash
hdfs namenode -format
```

Step 6: Start Hadoop Services 10. Start the Hadoop services using the following command:

```
bash
start-all.sh
```

Step 7: Verify Hadoop Installation 11. Check the running Hadoop processes using the `jps` command:

```
bash
jps
```

You should see a list of Java processes running, including `NameNode`, `DataNode`, `ResourceManager`, and `NodeManager`.

Step 8: Access Hadoop Web UI 12. Open a web browser and access the Hadoop Web UI at <http://localhost:50070/> (for HDFS) and <http://localhost:8088/> (for YARN ResourceManager).

You have successfully installed Hadoop on a single-node cluster. You can now use it for learning and experimenting with Hadoop and MapReduce.

4.2 DATA FORMAT - ANALYZING DATA WITH HADOOP

Data Formats in Hadoop:

- **Text Files:** Simple plain text files, where each line represents a record.

- **Sequence Files:** Binary files containing serialized key/value pairs.
- **Avro:** A data serialization system that provides rich data structures in a compact binary format.
- **Parquet:** A columnar storage file format optimized for use with Hadoop.

Analyzing Data with Hadoop:

- **MapReduce Programming Model:** Data analysis tasks in Hadoop are accomplished using the MapReduce programming model, where data is processed in two stages: the Map stage processes and sorts the data, and the Reduce stage performs summary operations.
- **Hive:** Hive is a data warehousing and SQL-like query language for Hadoop. It allows users to query and manage large datasets stored in Hadoop HDFS.
- **Pig:** Pig is a high-level platform and scripting language built on top of Hadoop, used for creating MapReduce programs for data analysis.

Analyzing data with Hadoop involves understanding the data format and structure, as well as using appropriate tools and techniques for processing and deriving insights from the data. Here are some key considerations when it comes to data format and analysis with Hadoop:

1. Data Format:

- **Structured Data:** If your data is structured, meaning it follows a fixed schema, you can use formats like Avro, Parquet, or ORC. These columnar storage formats are efficient for large-scale data analysis and support schema evolution.
- **Semi-Structured Data:** Data in JSON or XML format falls into this category. Hadoop can handle semi-structured data, and tools like Hive and Pig can help you query and process it effectively.
- **Unstructured Data:** Text data, log files, and other unstructured data can be processed using Hadoop as well. However, processing unstructured data often requires more complex parsing and natural language processing (NLP) techniques.

2. Data Ingestion:

- Before you can analyze data with Hadoop, you need to ingest it into the Hadoop Distributed File System (HDFS) or another storage system compatible with Hadoop. Tools like Apache Flume or Apache Sqoop can help with data ingestion.

3. Data Processing:

- Hadoop primarily uses the MapReduce framework for batch data processing. You write MapReduce jobs to specify how data should be processed. However, there are also high-level processing frameworks like Apache Spark and Apache Flink that provide more user-friendly abstractions and real-time processing capabilities.

4. Data Analysis:

- For SQL-like querying of structured data, you can use Apache Hive, which provides a SQL interface to Hadoop. Hive queries get translated into MapReduce or Tez jobs.
- Apache Pig is a scripting language specifically designed for data processing in Hadoop. It's useful for ETL (Extract, Transform, Load) tasks.
- For advanced analytics and machine learning, you can use Apache Spark, which provides MLlib for machine learning tasks, and GraphX for graph processing.

5. Data Storage and Compression:

- Hadoop provides various storage formats optimized for analytics (e.g., Parquet, ORC) and supports data compression to reduce storage requirements and improve processing speed.

6. Data Partitioning and Shuffling:

- Hadoop can automatically partition data into smaller chunks and shuffle it across nodes to optimize the processing pipeline.

7. Data Security and Access Control:

- Hadoop offers mechanisms for securing data and controlling access through authentication, authorization, and encryption.

8. Data Visualization:

- To make sense of the analyzed data, you can use data visualization tools like Apache Zeppelin or integrate Hadoop with business intelligence tools like Tableau or Power BI.

9. Performance Tuning:

- Hadoop cluster performance can be optimized through configuration settings and resource allocation. Understanding how to fine-tune these parameters is essential for efficient data analysis.

10. Monitoring and Maintenance:

- Regularly monitor the health and performance of your Hadoop cluster using tools like Ambari or Cloudera Manager. Perform routine maintenance tasks to ensure smooth operation.

Analyzing data with Hadoop involves a combination of selecting the right data format, processing tools, and techniques to derive meaningful insights from your data. Depending on your specific use case, you may need to choose different formats and tools to suit your needs.

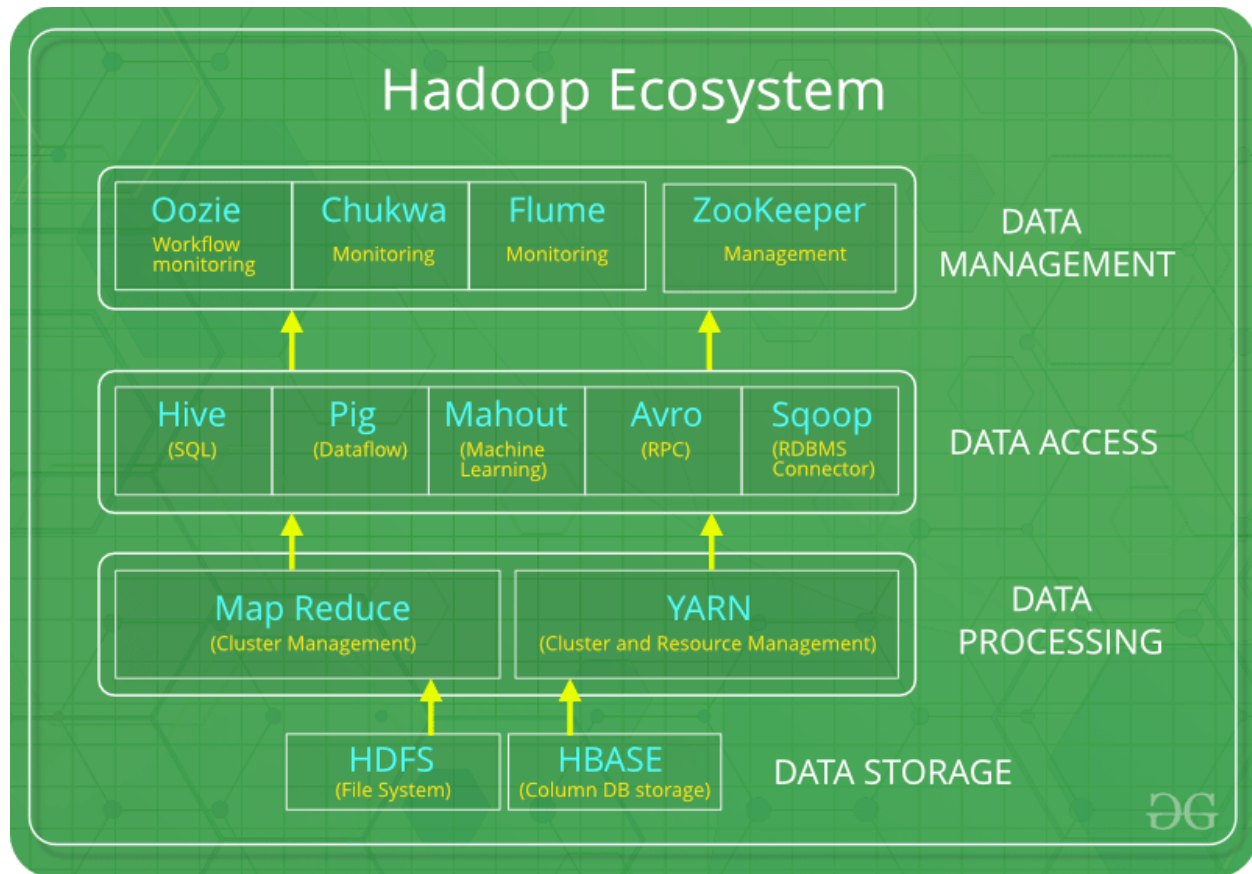
4.3 SCALING OUT

Scaling out is a fundamental concept in distributed computing and is one of the key benefits of using Hadoop for big data analysis. Here are some important points related to scaling out in Hadoop:

- **Horizontal Scalability:** Hadoop is designed for horizontal scalability, which means that you can expand the cluster by adding more commodity hardware machines to it. This allows you to accommodate larger datasets and perform more extensive data processing.
- **Data Distribution:** Hadoop's HDFS distributes data across multiple nodes in the cluster. When you scale out by adding more nodes, data is automatically distributed across these new machines. This distributed data storage ensures fault tolerance and high availability.
- **Processing Power:** Scaling out also means increasing the processing power of the cluster. You can run more MapReduce tasks and analyze data in parallel across multiple nodes, which can significantly speed up data processing.
- **Elasticity:** Hadoop clusters can be designed to be elastic, meaning you can dynamically add or remove nodes based on workload requirements. This is particularly useful in cloud-based Hadoop deployments where you pay for resources based on actual usage.
- **Balancing Resources:** When scaling out, it's important to consider resource management and cluster balancing. Tools like Hadoop YARN (Yet Another Resource Negotiator) help allocate and manage cluster resources efficiently.

Scaling Hadoop:

- **Horizontal Scaling:** Hadoop clusters can scale horizontally by adding more machines to the existing cluster. This approach improves processing power and storage capacity.
- **Vertical Scaling:** Vertical scaling involves adding more resources (CPU, RAM) to existing nodes in the cluster. However, there are limits to vertical scaling, and horizontal scaling is preferred for handling larger workloads.
- **Cluster Management Tools:** Tools like Apache Ambari and Cloudera Manager help in managing and scaling Hadoop clusters efficiently.
- **Data Partitioning:** Proper data partitioning strategies ensure that data is distributed evenly across the cluster, enabling efficient processing.



4.4. HADOOP STREAMING

What is Hadoop Streaming? Hadoop Streaming is a utility that comes with Hadoop distribution. It allows you to create and run MapReduce jobs with any executable or script as the mapper and/or the reducer. This means you can use any programming language that can read from standard input and write to standard output for your MapReduce tasks.

How Hadoop Streaming Works:

1. **Input:** Hadoop Streaming reads input from HDFS or any other file system and provides it to the mapper as lines of text.
2. **Mapper:** You can use any script or executable as a mapper. Hadoop Streaming feeds the input lines to the mapper's standard input.
3. **Shuffling and Sorting:** The output from the mapper is sorted and partitioned by the Hadoop framework.

4. **Reducer:** Similarly, you can use any script or executable as a reducer. The reducer reads sorted input lines from its standard input and produces output, which is written to HDFS or any other file system.
5. **Output:** The final output is stored in HDFS or the specified output directory.

Advantages of Hadoop Streaming:

- **Language Flexibility:** It allows developers to use languages like Python, Perl, Ruby, etc., for writing MapReduce programs, extending Hadoop's usability beyond Java developers.
- **Rapid Prototyping:** Developers can quickly prototype and test algorithms without the need to compile and package Java code.

4.5 HADOOP PIPES

What is Hadoop Pipes? Hadoop Pipes is a C++ API to implement Hadoop MapReduce applications. It enables the use of C++ to write MapReduce programs, allowing developers proficient in C++ to leverage Hadoop's capabilities.

How Hadoop Pipes Works:

1. **Mapper and Reducer:** Developers write the mapper and reducer functions in C++.
2. **Input:** Hadoop Pipes reads input from HDFS or other file systems and provides it to the mapper as key-value pairs.
3. **Map and Reduce Operations:** The developer specifies the map and reduce functions, defining the logic for processing the input key-value pairs.
4. **Output:** The final output is written to HDFS or another file system.

Advantages of Hadoop Pipes:

- **Performance:** Programs written in C++ can sometimes be more performant due to the lower-level memory management and execution speed of C++.
- **C++ Libraries:** Developers can leverage existing C++ libraries and codebases, making it easier to integrate with other systems and tools.

Both Hadoop Streaming and Hadoop Pipes provide flexibility in terms of programming languages, enabling a broader range of developers to work with Hadoop and leverage its powerful data processing capabilities.

4.6 DESIGN OF HADOOP DISTRIBUTED FILE SYSTEM (HDFS):

Architecture: Hadoop Distributed File System (HDFS) is designed to store very large files across multiple machines in a reliable and fault-tolerant manner. Its architecture consists of the following components:

1. **NameNode:** The NameNode is the master server that manages the namespace and regulates access to files by clients. It stores metadata about the files and directories, such as the file structure tree and the mapping of file blocks to DataNodes.
2. **DataNode:** DataNodes are responsible for storing the actual data. They store data in the form of blocks and send periodic heartbeats and block reports to the NameNode to confirm that they are functioning correctly.
3. **Block:** HDFS divides large files into fixed-size blocks (typically 128 MB or 256 MB). These blocks are stored across the DataNodes in the cluster.
4. **Secondary NameNode (Deprecated Term):** The Secondary NameNode is not a backup or failover NameNode. Its primary function is to periodically merge the namespace image and edit log files, reducing the load on the primary NameNode.

Replication: HDFS replicates each block multiple times (usually three) and places these replicas on different DataNodes across the cluster. Replication ensures fault tolerance. If a DataNode or block becomes unavailable, the system can continue to function using the remaining replicas.

4.7 HDFS CONCEPTS:

1. **Block:** Blocks are the fundamental units of data storage in HDFS. Each file is broken down into blocks, and these blocks are distributed across the cluster's DataNodes.
2. **Namespace:** HDFS organizes files and directories in a hierarchical namespace. The NameNode manages this namespace and regulates access to files by clients.
3. **Replication:** As mentioned earlier, HDFS replicates blocks for fault tolerance. The default replication factor is 3, but it can be configured based on the cluster's requirements.
4. **Fault Tolerance:** HDFS achieves fault tolerance by replicating data blocks across multiple nodes. If a DataNode or block becomes unavailable due to hardware failure or other issues, the system can continue to operate using the replicated blocks.
5. **High Write Throughput:** HDFS is optimized for high throughput of data, making it suitable for applications with large datasets. It achieves this through the parallelism of writing and reading data across multiple nodes.
6. **Scalability:** HDFS is designed to scale horizontally by adding more nodes to the cluster. This scalability allows Hadoop clusters to handle large and growing amounts of data.

7. Data Integrity: HDFS ensures data integrity by storing checksums of data with each block. This checksum is verified by clients and DataNodes to ensure that data is not corrupted during storage or transmission.

4.8 JAVA INTERFACE IN HADOOP:

Hadoop provides Java APIs that developers can use to interact with the Hadoop ecosystem. The Java interface in Hadoop includes various classes and interfaces that allow developers to create MapReduce jobs, configure Hadoop clusters, and manipulate data stored in HDFS. Here's a brief overview of key components in the Java interface:

1. **org.apache.hadoop.mapreduce Package:**
 - Mapper: Interface for the mapper task in a MapReduce job.
 - Reducer: Interface for the reducer task in a MapReduce job.
 - Job: Represents a MapReduce job configuration.
 - InputFormat: Specifies the input format of the job.
 - OutputFormat: Specifies the output format of the job.
 - Configuration: Represents Hadoop configuration properties.
2. **org.apache.hadoop.fs Package:**
 - FileSystem: Interface representing a file system in Hadoop (HDFS, local file system, etc.).
 - Path: Represents a file or directory path in Hadoop.
3. **org.apache.hadoop.io Package:**
 - Writable: Interface for custom Hadoop data types.
 - WritableComparable: Interface for custom data types that are comparable and writable.

Developers use these interfaces and classes to create custom MapReduce jobs, configure input and output formats, and interact with HDFS. They can implement the `Mapper` and `Reducer` interfaces to define their own map and reduce logic for processing data.

4.9 DATA FLOW IN HADOOP:

Data flow in Hadoop refers to the movement of data between different stages of a MapReduce job or between Hadoop components. Here's how data flows in a typical Hadoop MapReduce job:

1. **Input Phase:**
 - Input data is read from one or more sources, such as HDFS files, HBase tables, or other data storage systems.
 - Input data is divided into input splits, which are processed by individual mapper tasks.
2. **Map Phase:**
 - Mapper tasks process the input splits and produce intermediate key-value pairs.
 - The intermediate data is partitioned, sorted, and grouped by key before being sent to the reducers.

3. **Shuffle and Sort Phase:**

- Intermediate data from all mappers is shuffled and sorted based on keys.
- Data with the same key is grouped together, and each group of data is sent to a specific reducer.

4. **Reduce Phase:**

- Reducer tasks receive sorted and grouped intermediate data.
- Reducers process the data and produce the final output key-value pairs, which are typically written to HDFS or another storage system.

5. **Output Phase:**

- The final output is stored in HDFS or another output location specified by the user.
- Users can access the output data for further analysis or processing.

4.10 HADOOP I/O - DATA INTEGRITY:

Ensuring data integrity is crucial in any distributed storage and processing system like Hadoop. Hadoop provides several mechanisms to maintain data integrity:

1. **Replication:**

- HDFS stores multiple replicas of each block across different nodes. If a replica is corrupted, Hadoop can use one of the other replicas to recover the lost data.

2. **Checksums:**

- HDFS uses checksums to validate the integrity of data blocks. Each block is associated with a checksum, which is verified by both the client reading the data and the DataNode storing the data. If a block's checksum doesn't match the expected value, Hadoop knows the data is corrupted and can request it from another node.

3. **Write Pipelining:**

- HDFS pipelines the data through several nodes during the writing process. Each node in the pipeline verifies the checksums before passing the data to the next node. If a node detects corruption, it can request the block from another replica.

4. **Error Detection and Self-healing:**

- Hadoop can detect corrupted blocks and automatically replace them with healthy replicas from other nodes, ensuring the integrity of the stored data.

4.11 COMPRESSION AND SERIALIZATION IN HADOOP:

1. **Compression:**

- Hadoop supports various compression algorithms like Gzip, Snappy, and LZO. Compressing data before storing it in HDFS can significantly reduce storage requirements and improve the efficiency of data processing. You can specify the compression codec when writing data to HDFS or when configuring MapReduce jobs.

Example of specifying a compression codec in a MapReduce job:

```
java
conf.set("mapreduce.map.output.compress", "true");
conf.set("mapreduce.map.output.compress.codec",
"org.apache.hadoop.io.compress.SnappyCodec");
```

Serialization:

- Hadoop uses its own serialization framework called Writable to serialize data efficiently. Writable data types are Java objects optimized for Hadoop's data transfer. You can also use Avro or Protocol Buffers for serialization. These serialization formats are more efficient than Java's default serialization mechanism, especially in the context of large-scale data processing.

Example of using Avro for serialization:

```
java
// Writing Avro data to HDFS
DatumWriter<YourAvroRecord> datumWriter = new
SpecificDatumWriter<>(YourAvroRecord.class);
DataFileWriter<YourAvroRecord> dataFileWriter =new
DataFileWriter<>(datumWriter);
dataFileWriter.create(yourAvroRecord.getSchema(), new File("output.avro"));
dataFileWriter.append(yourAvroRecord);
dataFileWriter.close();
```

By utilizing these mechanisms, Hadoop ensures that data integrity is maintained during storage and processing. Additionally, compression and efficient serialization techniques optimize storage and data transfer, contributing to the overall performance of Hadoop applications.

4.12 AVRO - FILE-BASED DATA STRUCTURES:

Apache Avro is a data serialization framework that provides efficient data interchange in Hadoop. It enables the serialization of data structures in a language-independent way, making it ideal for data stored in files. Avro uses JSON for defining data types and protocols, allowing data to be self-describing and allowing complex data structures.

Key Concepts:

1. **Schema Definition:** Avro uses JSON to define schemas. Schemas define the data structure, including types and their relationships. For example, you can define records, enums, arrays, and more in Avro schemas.

```
json
{
  "type": "record",
  "name": "User",
  "fields": [
    { "name": "name", "type": "string" },
    { "name": "age", "type": "int" },
    { "name": "address", "type": "string" }
  ]
}
```

```
}  
{
```

- 1.
2. **Serialization:** Avro encodes data using the defined schema, producing compact binary files. Avro data is self-describing, meaning that the schema is embedded in the data itself.
3. **Deserialization:** Avro can deserialize the data back into its original format using the schema information contained within the data.
4. **Code Generation:** Avro can generate code in various programming languages from a schema. This generated code helps in working with Avro data in a type-safe manner.

Avro is widely used in the Hadoop ecosystem due to its efficiency, schema evolution capabilities, and language independence, making it a popular choice for serializing data in Hadoop applications.

4.13 CASSANDRA - HADOOP INTEGRATION:

Apache Cassandra is a highly scalable, distributed NoSQL database that can handle large amounts of data across many commodity servers. Integrating Cassandra with Hadoop provides the ability to combine the advantages of a powerful database system with the extensive data processing capabilities of the Hadoop ecosystem.

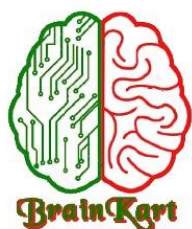
Integration Strategies:

1. **Cassandra Hadoop Connector:** Cassandra provides a Hadoop integration tool called the Cassandra Hadoop Connector. It allows MapReduce jobs to read and write data to and from Cassandra.
2. **Cassandra as a Source or Sink:** Cassandra can act as a data source or sink for Apache Hadoop and Apache Spark jobs. You can configure Hadoop or Spark to read data from Cassandra tables or write results back to Cassandra.
3. **Cassandra Input/Output Formats:** Cassandra supports Hadoop Input/Output formats, allowing MapReduce jobs to directly read from and write to Cassandra tables.

Benefits of Integration:

- **Data Processing:** You can perform complex data processing tasks on data stored in Cassandra using Hadoop's distributed processing capabilities.
- **Data Aggregation:** Aggregate data from multiple Cassandra nodes using Hadoop's parallel processing, enabling large-scale data analysis.
- **Data Export and Import:** Use Hadoop to export data from Cassandra for backup or analytical purposes. Similarly, you can import data into Cassandra after processing it using Hadoop.

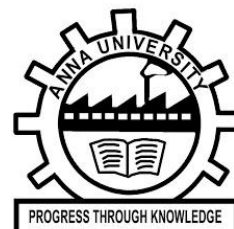
Integrating Cassandra and Hadoop allows businesses to leverage the best of both worlds: Cassandra's real-time, high-performance database capabilities and Hadoop's extensive data processing and analytics features. This integration enables robust, large-scale data applications for a variety of use cases.



www.BrainKart.com

Anna University

for Affiliated Engineering College - 2021 Regulation



AID (Artificial Intelligence & Data Science Engineering)

1st Semester ▶

2nd Semester ▶

3rd Semester ▶

4th Semester ▶

5th Semester ▶

6th Semester ▶

7th Semester ▶

8th Semester ▶

Click on Subject/Paper under Semester to enter.

1st Semester

[Professional English - I - HS3152](#)

[Matrices and Calculus - MA3151](#)

[Engineering Physics - PH3151](#)

[Engineering Chemistry - CY3151](#)

[Problem Solving and Python Programming - GE3151](#)

2nd Semester

[Professional English - II - HS3252](#)

[Statistics and Numerical Methods - MA3251](#)

[Engineering Graphics - GE3251](#)

[Physics for Information Science - PH3256](#)

[Basic Electrical and Electronics Engineering - BE3251](#)

[Data Structures Design - AD3251](#)

3rd Semester

[Discrete Mathematics - MA3354](#)

[Digital Principles and Computer Organization - CS3351](#)

[Database Design and Management - AD3391](#)

[Design and Analysis of Algorithms - AD3351](#)

[Data Exploration and Visualization - AD3301](#)

[Artificial Intelligence - AL3391](#)

4th Semester

[Environmental Sciences and Sustainability - GE3451](#)

[Probability and Statistics - MA3391](#)

[Operating Systems - AL3452](#)

[Machine Learning - AL3451](#)

[Fundamentals of Data Science and Analytics - AD3491](#)

[Computer Networks - CS3591](#)

5th Semester

[Deep Learning - AD3501](#)

[Data and Information Security - CW3551](#)

[Distributed Computing - CS3551](#)

[Big Data Analytics - CCS334](#)

[Elective 1](#)

[Elective 2](#)

6th Semester

[Embedded Systems and IoT - CS3691](#)

[Open Elective-1](#)

[Elective-3](#)

[Elective-4](#)

[Elective-5](#)

[Elective-6](#)

7th Semester

[Human Values and Ethics - GE3791](#)

[Open Elective 2](#)

[Open Elective 3](#)

[Open Elective 4](#)

[Management Elective](#)

8th Semester

[Project Work / Internship](#)



Anna University Notes

Therithal Info
Contains ads

3.7★

199 reviews

50K+

Downloads

3+

Rated for 3+ ©

Install



BrainKart: Learning, Study App

Therithal Info
Contains ads

4.5★

160 reviews

10K+

Downloads

3+

Rated for 3+ ©

Install

All Computer Engg Subjects - [B.E., M.E.,]

(Click on Subjects to enter)

<u>Programming in C</u>	<u>Computer Networks</u>	<u>Operating Systems</u>
<u>Programming and Data Structures I</u>	<u>Programming and Data Structure II</u>	<u>Problem Solving and Python Programming</u>
<u>Database Management Systems</u>	<u>Computer Architecture</u>	<u>Analog and Digital Communication</u>
<u>Design and Analysis of Algorithms</u>	<u>Microprocessors and Microcontrollers</u>	<u>Object Oriented Analysis and Design</u>
<u>Software Engineering</u>	<u>Discrete Mathematics</u>	<u>Internet Programming</u>
<u>Theory of Computation</u>	<u>Computer Graphics</u>	<u>Distributed Systems</u>
<u>Mobile Computing</u>	<u>Compiler Design</u>	<u>Digital Signal Processing</u>
<u>Artificial Intelligence</u>	<u>Software Testing</u>	<u>Grid and Cloud Computing</u>
<u>Data Ware Housing and Data Mining</u>	<u>Cryptography and Network Security</u>	<u>Resource Management Techniques</u>
<u>Service Oriented Architecture</u>	<u>Embedded and Real Time Systems</u>	<u>Multi - Core Architectures and Programming</u>
<u>Probability and Queueing Theory</u>	<u>Physics for Information Science</u>	<u>Transforms and Partial Differential Equations</u>
<u>Technical English</u>	<u>Engineering Physics</u>	<u>Engineering Chemistry</u>
<u>Engineering Graphics</u>	<u>Total Quality Management</u>	<u>Professional Ethics in Engineering</u>
<u>Basic Electrical and Electronics and Measurement Engineering</u>	<u>Problem Solving and Python Programming</u>	<u>Environmental Science and Engineering</u>

